Version: 1.0



Selection

C-Programming-Fundamentals

Résumé

Implémentez des fonctions fondamentales en C pour afficher des caractères, l'alphabet, des chiffres, et vérifier le signe d'un entier via la sortie standard.

#C #Fundamentals #Output



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Tutoriel 6

4 Exercice 0 : ft_putchar 9

5 Exercice 1 : ft_print_alphabet 10

6 Exercice 2 : ft_print_reverse_alphabet 11

7 Exercice 3 : ft_print_numbers 12

8 Exercice 4 : ft_is_negative 13

9 Soumission et évaluation par les pairs 14

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

L'huile de foie de morue est un complément alimentaire dérivé du foie de morue (poisson de la famille des Gadidae).

Comme la plupart des huiles de poisson, elle contient des taux élevés d'acides gras oméga-3 : acide eicosapentaénoïque (EPA) et acide docosahexaénoïque (DHA). L'huile de foie de morue contient également les vitamines A et D.

Elle a été historiquement consommée pour son apport en vitamines A et D.

On l'administrait autrefois couramment aux enfants, car il a été démontré que la vitamine D prévient le rachitisme et d'autres symptômes de carence.

Contrairement à l'huile de foie de morue, le C est bon : mangez-en !

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Tutoriel

Bienvenue dans votre premier tutoriel de programmation en **C** ! Avant de plonger dans le code, vous devez comprendre les bases. Regardez la vidéo suivante sur les langages de programmation et la mémoire de l'ordinateur : Le langage de programmation C. Faites attention à la manière dont les ordinateurs stockent et manipulent les données.

- Comprendre la **mémoire** et le fonctionnement des programmes est crucial avant d'écrire votre première ligne de code **C**.
- Après avoir regardé la vidéo, créez votre premier programme `C`. En suivant l'exemple de la vidéo, créez une fonction `main` vide dans un fichier appelé `first_program.c` :

first_program.c

```
int main()
{
    return (0);
}
```

- Compilez-le en utilisant la commande suivante : `cc first_program.c -o first_program`
- Exécutez-le avec la commande `./first_program`. Que se passe-t-il ?
- Rien de visible ne se passe ? C'est normal. Vous n'avez pas encore d'instructions ! Modifiez votre code et ajoutez une instruction simple comme `1+1;` à l'intérieur de la fonction `main`. Compilez et exécutez-le à nouveau.
- Toujours rien de visible ? En fait, votre code a fait quelque chose, mais vous ne le "voyez" tout simplement pas. L'ordinateur a calculé `1+1=2`, mais n'a rien affiché. Il est temps de rendre quelque chose visible !
- Vous devez afficher quelque chose dans le terminal. Créez la fonction `ft_putchar` qui affiche un caractère :

© 2026 Association 42. - Tous droits réservés

first_program.c

```
#include <unistd.h>

void ft_putchar(char c)
{
    write(1, &c, 1);
}
```

- Maintenant, créez un programme complet qui affiche un caractère. Appelez `ft_putchar` depuis votre fonction `main`. Le paramètre `char` est un code `ASCII` (un nombre entre 0 et 127 qui représente un caractère).
- Essayez d'afficher différents caractères :

first_program.c

```
ft_putchar('A');
ft_putchar('B');
ft_putchar('*');
```

Que se passe-t-il ?



Les codes ASCII sont la manière dont les ordinateurs représentent les caractères sous forme de nombres. 'A' est 65, 'a' est 97, '0' est 48, etc. Avec l'aide de vos pairs, essayez de trouver la commande shell qui affiche la table ASCII dans votre terminal.

- Maintenant, utilisons une variable ! Définissez une variable `char`, attribuez-lui une valeur de caractère `ASCII`, et affichez-la en utilisant `ft_putchar` :

first_program.c

```
char my_char;
my_char = 'X';
ft_putchar(my_char);
```

- Essayez d'ajouter un à la variable lors de l'appel de `ft_putchar` :

© 2026 Association 42. - Tous droits réservés

first_program.c

```
char my_char;
my_char = 'X';
ft_putchar(my_char + 1);
```

Quel `caractère` apparaît et pourquoi ?

- Maintenant, vous avez besoin d'une boucle ! Comment pourriez-vous répéter 10 fois l'affichage d'un `caractère` ? Recherchez les **variables de compteur** et la structure **while** en ligne. Essayez d'afficher le même `caractère` 10 fois de suite.
- Créez une boucle qui affiche différents `caractères`. Par exemple, affichez les lettres de `A` à `J` en utilisant une variable de compteur que vous ajoutez à une valeur de base `ASCII`.
- Les boucles sont fondamentales en programmation. Elles vous permettent de répéter des actions sans écrire le même code plusieurs fois.
- Expérimentez avec différents motifs de boucle : comptez à rebours, comptez à l'envers, affichez des motifs comme des étoiles ou des nombres...
- Vous êtes prêt pour les exercices ! Vous allez expérimenter avec diverses **boucles**, **branches conditionnelles** (`if, else`), et les mélanger ensemble. N'oubliez pas de demander de l'aide à vos pairs et de discuter de votre code.

  La clé pour apprendre la programmation est la pratique, l'expérimentation et la discussion avec les pairs. N'hésitez pas à essayer différentes approches !

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Exercice 0 : ft_putchar

	Exercice: 0	
ft_putchar		
Répertoire: *ex0/*		
Fichiers à Rendre: `ft_putchar.c`		
Autorisé: `write`		

- Écrivez une fonction qui affiche le caractère passé en paramètre.

  Prototype :

  ```
  void ft_putchar(char c);
  ```

- Pour afficher le caractère, vous devez utiliser la fonction `write` comme suit.

  Prototype :

  ```
  write(1, &c, 1);
  ```



Le premier délai de tentative est court, n'hésitez pas à déclencher une évaluation intermédiaire pour mesurer vos progrès.

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 1 : ft_print_alphabet

	Exercice: 1	
ft_print_alphabet		
Répertoire: *ex1/*		
Fichiers à Rendre: `ft_print_alphabet.c`		
Autorisé: `write`		

- Créez une fonction qui affiche l'alphabet en minuscules, sur une seule ligne, par ordre croissant, en commençant par la lettre 'a'.

Prototype :

```
void ft_print_alphabet(void);
```



N'hésitez pas à interroger quelqu'un au hasard dans votre cluster.

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Exercice 2 : ft_print_reverse_alphabet

	Exercice: 2	
ft_print_reverse_alphabet		
Répertoire: *ex2/*		
Fichiers à Rendre: `ft_print_reverse_alphabet.c`		
Autorisé: `write`		

- Créez une fonction qui affiche l'alphabet en minuscules, sur une seule ligne, par ordre décroissant, en commençant par la lettre 'z'.

Prototype :

```
void ft_print_reverse_alphabet(void);
```



Faites un `git push` régulièrement.

© 2026 Association 42. - Tous droits réservés

Chapitre 7

Exercice 3 : ft_print_numbers

	Exercice: 3	
ft_print_numbers		
Répertoire: *ex3/*		
Fichiers à Rendre: `ft_print_numbers.c`		
Autorisé: `write`		

- Créez une fonction qui affiche tous les chiffres, sur une seule ligne, par ordre croissant.

Prototype :

```
void ft_print_numbers(void);
```



La collaboration est la clé du succès.

© 2026 Association 42. - Tous droits réservés

Chapitre 8

Exercice 4 : ft_is_negative

	Exercice: 4	
ft_is_negative		
Répertoire: *ex*4/		
Fichiers à Rendre: `ft_is_negative.c`		
Autorisé: `write`		

- Créez une fonction qui affiche 'N' ou 'P' selon le signe de l'entier passé en paramètre. Si `n` est négatif, affichez 'N'. Si `n` est positif ou nul, affichez 'P'.

Prototype :

```
void ft_is_negative(int n);
```



L'échec fait partie de votre parcours d'apprentissage.

© 2026 Association 42. - Tous droits réservés

Chapitre 9

Soumission et évaluation par les pairs

Déposez votre devoir dans votre dépôt `Git` comme d'habitude. Seul le contenu présent dans votre dépôt sera évalué lors de la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous devez rendre uniquement les fichiers demandés par l'énoncé de ce projet.

© 2026 Association 42. - Tous droits réservés